

Competition Report

Chest X-ray Classification

Ersa

1. Overview

This project targets 5-class chest X-ray classification using a dense-only neural network design. The final model uses a patch-based representation of the image and mixes information across patches and channels through MLP-style blocks. The main objective was to improve macro F1-score while keeping the training and inference process reliable.

2. Final Model Idea

The final architecture follows this flow:

- Convert input image to grayscale and normalize.
- Resize to a fixed working size and split into non-overlapping patches.
- Apply a shared dense projection to each patch.
- Add learnable positional embeddings so the model keeps spatial context.
- Use token-mixing and channel-mixing MLP blocks with skip connections. I got this idea of skipping connections and channel-mixing from this [Github repo](#).
- Aggregate features with global pooling and predict with a final softmax layer. silhouette within the chest.

This design was chosen to capture both local texture and global chest structure without convolution. Since I was considering the problem in a medical context, the changes needed to be visible to the naked eye and primarily involve global features. For example, large-scale changes such as consolidation or an enlarged cardiac

3. Training Strategy and Improvements

Key ideas that improved the score:

- **Multi-network in parallel:** I tried this strategy, but it did not lead to a significant improvement in the F1 score; therefore, I decided to omit the use of multiple layers. To be more accurate, I focused on downsampling within each network (each branch of Multi-network in parallel) so that each network would see a more diverse subset of the samples. In this setup, each parallel network was designed to have the same number of samples for each class. In theory, this approach should have worked well; however, it did not produce the expected results. This may be due either to an incorrect implementation on my part or to limitations inherent to the idea itself. I tested this approach at the end of the competition (last day) using a simple r network for each branch and added additional dropout and regularization to mitigate overfitting, but the performance still did not improve.

- **Class-aware training:** Class-balanced sampling and class weighting were used to reduce collapse on minority classes.

My approach started by increasing the frequency of class 5 samples within each batch, effectively enforcing aggressive class balancing so that all classes were equally represented during training. In theory, this should have helped the network pay more attention to the rare class; however, in practice, performance plateaued and could not be improved further. This led to a clear trade-off between “*pay attention to the rare class*” (via class weighting and oversampling) and “*do not memorize it*” (via regularization and dropout). As shown in the Results section, the model performs reasonably well on the training set—though it clearly overfits on class 5—but generalizes poorly to the validation set. Specifically, the F1 score for class 5 drops from 0.3849 during training to 0.1548 on validation, which is significantly worse compared to other classes and indicates strong overfitting. Increasing the dropout rate helped reduce memorization of class 5, but at the cost of degrading performance across the other classes, leading to an overall loss in accuracy. This trade-off dominated the model development process, and approximately 90% of the total effort was spent attempting to balance these competing effects.

I believe this stage became the main bottleneck of the model. It is likely that the winner of the competition relied on a more effective strategy at this step.

More technical view of my final setting:

- I used `BALANCE_TEMPERATURE = 0.6` in the balanced data pipeline. The sampling probability per class is proportional to

$$P(c) \propto n_c^T, \quad T = 0.6$$

where n_c is the number of samples in class c . Interpretation: $T = 1$ keeps the original class distribution, $T = 0$ makes all classes equally likely, and $0 < T < 1$ is a partial-balance regime. So $T = 0.6$ was my compromise between full balancing and the original data distribution.

- I also used `CLASS_WEIGHT_MAP = {0:1.0, 1:1.1, 2:1.0, 3:1.05, 4:1.7}`. Apparently this modifies the loss, as

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N w_{y_i} \text{CE}(y_i, \hat{y}_i),$$

so errors on class 5 ($w = 1.7$) contribute much more than errors on majority classes. If I explain this using what we learned in NN class last semester, the way I interpret it is that class weighting effectively changes the gradient landscape: it creates “steeper slopes” toward learning the higher-weight classes. But if the weight is too large, like the large cross-entropy weight I used for class 5, it can also push the model to overfit that class.

- In practice, these two mechanisms worked together: sampling made minority examples appear more often, and class weights amplified their gradient signal. This improved minority sensitivity, but if pushed too hard, it also increased overfitting risk on class 5.
- **F1-driven monitoring:** Custom callbacks were used to report per-class F1 scores for both the training and validation sets, which made weak classes visible early in the training process. This allowed me to monitor overfitting per-class and directly compare each class’s training F1 with its corresponding validation F1.
- **Mild augmentation:** Horizontal flipping and small rotations were used as data augmentation techniques to improve robustness without excessively distorting medically meaningful patterns. While the precise impact of these augmentations is not entirely clear to me, results

showed that horizontal flipping in particular was beneficial, which is reasonable given the left–right symmetry commonly present in X-ray images.

- **Stable optimization:** AdamW with LR scheduling and early stopping helped keep training stable. I used `ReduceLROnPlateau` with factor 0.5, patience = 5, and `min_lr` = 10^{-6} , monitored on validation macro-F1. When validation F1 stopped improving for several epochs, the learning rate was reduced by half:

$$\eta \leftarrow 0.5 \eta.$$

This was useful for two reasons: first, the model could learn quickly in early epochs with a larger step size; second, once it reached a plateau, smaller steps helped avoid oscillation and improved convergence along the smaller eigenvalues. This is similar to the lab we saw in NN class, where a large learning rate causes the optimization to fluctuate perpendicular to the dominant gradient contour lines. Without this scheduler, I repeatedly observed unstable late-epoch behavior, especially under imbalance-aware training. I realized the learning rate scheduler matters when the problem becomes complicated, like deep learning networks.

4. Results

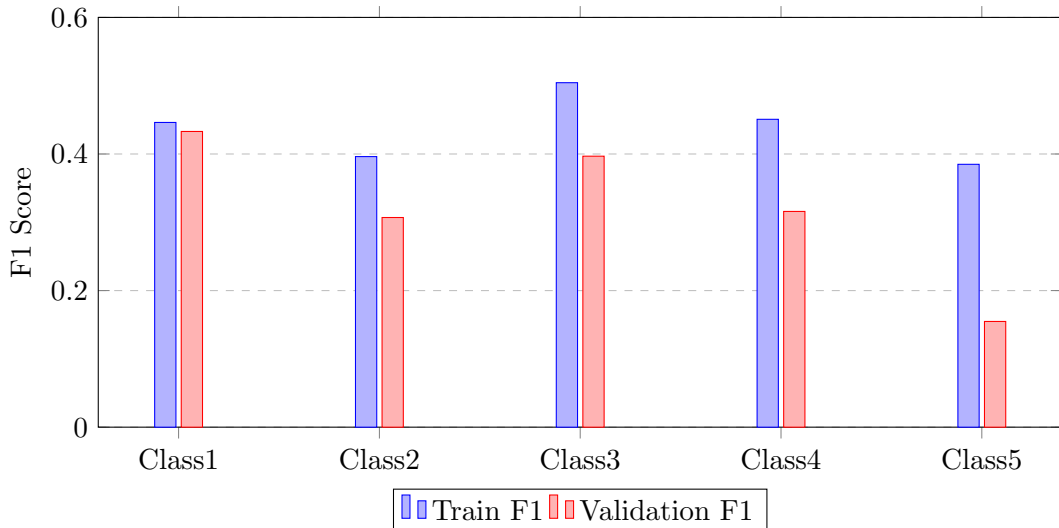
4.1 Per-Class F1 (Current Run)

Class	Train F1	Validation F1
Class 1	0.4462	0.4330
Class 2	0.3962	0.3070
Class 3	0.5044	0.3968
Class 4	0.4508	0.3159
Class 5	0.3849	0.1548
Macro F1	0.4365	0.3215

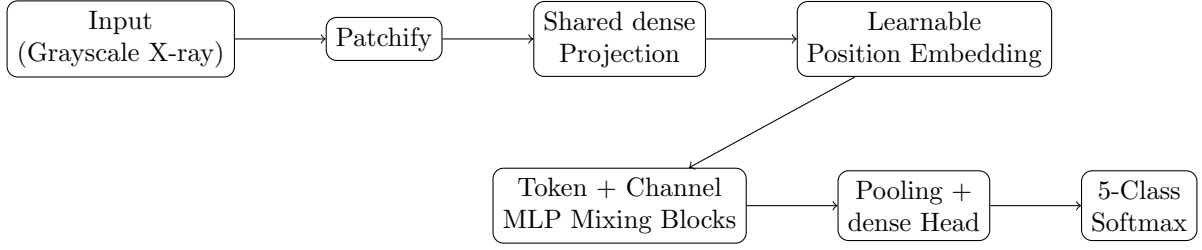
4.2 Final Test Metric

- Test macro F1-score: **0.3449**
- Competition F1-score: **0.327**

4.3 F1 Visualization



5. Architecture Diagram (High Level)



In the final implementation, I used the following components and each one had a specific role:

- **Resize to 120×120** — Why: reduce memory/computation while keeping global chest anatomy visible.
- **8×8 non-overlapping patches (225 tokens)** — Why: convert the image into local regions so dense layers can learn local-to-global patterns.
- **Shared dense projection to 128 dims** — Why: learn a compact feature representation per patch with shared parameters.
- **Learnable positional embedding** — Why: preserve location context (upper/lower lung, heart region, diaphragm area). I had this idea but I could not implement it, so I relied on AI. However, I did not fully understand the resulting approach, and I was unable to properly validate the code by AI.
- **3 mixer blocks** — Why: enough depth for feature interaction without making optimization unstable.
- **Token-mixing MLP (64 hidden)** — Why: exchange information across patches (global structure reasoning).
- **Channel-mixing MLP (256 hidden)** — Why: enrich feature transformation within each patch token.
- **Dense-style skip connectivity (concat + projection)** — Why: improve gradient flow, reuse earlier features, and stabilize training.
- **Global average pooling** — Why: summarize token-level information into one image-level descriptor. I just love average and max pooling.
- **Dense(256) + Dropout + Softmax(5)** — Why: final nonlinear decision layer, regularization, and class probability output.

6. Discussion

The best gains came from combining three elements: (1) patch-based dense modeling with positional information, (2) class-aware training for imbalance, and (3) per-class F1 monitoring for targeted iteration. This workflow improved macro F1 and made failure modes clearer, especially for hard minority cases.

7. Conclusion

The final system achieved competitive macro F1 with a compact and interpretable dense-only design. Future gains are likely from better minority-class generalization and calibration.

8. Fun Note

One funny point about my nickname **Ersa**: Named by Twitter Contest. :)

In 2019, the Carnegie Institution for Science ran a Twitter naming contest for five newly discovered moons of Jupiter. Specifically a school in Canada, suggested Ersa, which was approved by the International Astronomical Union.